

Lab 2.1 — Variables and Data Types

Objective

By the end of this file, you will be able to declare variables in Python, assign values of different data types (`str`, `int`, `float`, `bool`), display variable contents using `print()`, and inspect data types using `type()`.

Background

What is a Variable?

A variable is a **named container** that holds a value in memory. Think of it like a labelled box: you put something inside, and whenever you use the label, Python gives you the contents.

In many programming languages (like C or Java), you must declare a variable's type before using it. **Python is dynamically typed** — you simply assign a value and Python figures out the type automatically.

Why This Matters in IT & Automation

Variables are the foundation of every program. Whether you are automating a network scan, processing a CSV report, or querying an API, you constantly store data in variables — IP addresses (strings), port numbers (integers), response times (floats), and flags (booleans). Understanding data types prevents subtle bugs like adding a number to a string or comparing values of the wrong type.

Real-World Examples

Variable	Type	Example Value
Username in a login system	<code>str</code>	<code>"admin"</code>
Port number in a socket connection	<code>int</code>	<code>8080</code>
CPU usage percentage	<code>float</code>	<code>87.5</code>
Is the server online?	<code>bool</code>	<code>True</code>

Lab Overview

Objectives	Declare variables; work with <code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> ; use <code>type()</code>
Estimated Time	15–20 minutes
Prerequisites	Lab 1 complete; comfortable running <code>.py</code> files

Original Material

A variable is a named container for storing a value. In Python, you create a variable simply by assigning it a value — no declaration keyword needed.

Code

```
# Assigning values to variables
name = "Alice"
age = 30
height = 5.6
is_student = True

# Displaying variable contents
print(name)
print(age)
print(height)
print(is_student)

# Checking the type of a variable
print(type(name))
print(type(age))
print(type(height))
print(type(is_student))
```

Expected Output

```
Alice
30
5.6
True
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Code Execution

1. Python reads line 2: allocates memory, stores the string "Alice", and binds the name `name` to it.
2. Lines 3–5 repeat the same process for `age` (integer 30), `height` (float 5.6), and `is_student` (boolean True).
3. Lines 8–11 call `print()` four times — each call retrieves the value stored under that variable name and writes it to the console.

4. Lines 14–17 call `type()` on each variable. `type()` returns a **class object** (e.g., `<class 'str'>`), which `print()` then displays.

Step-by-Step Explanation

Line	Code	What Happens
2	<code>name = "Alice"</code>	Creates a <code>str</code> variable called <code>name</code> holding "Alice"
3	<code>age = 30</code>	Creates an <code>int</code> variable called <code>age</code> holding 30
4	<code>height = 5.6</code>	Creates a <code>float</code> variable called <code>height</code> holding 5.6
5	<code>is_student = True</code>	Creates a <code>bool</code> variable called <code>is_student</code> holding True
8–11	<code>print(name) ...</code>	Outputs the stored value to the console
14–17	<code>print(type(name)) ...</code>	Outputs the data type class of each variable

Key Concepts

Concept	Definition
Variable	A named label bound to a value in memory
<code>str</code>	Text values enclosed in single or double quotes: "Alice"
<code>int</code>	Whole numbers (no decimal point): 30, -5, 1000
<code>float</code>	Numbers with a decimal point: 5.6, 3.14, -0.5
<code>bool</code>	Logical values — only True or False (capital first letter)
<code>type()</code>	Built-in function that returns the data type of any object
Assignment operator <code>=</code>	Binds a name (left) to a value (right) — not mathematical equality

Common Mistakes

WRONG — Using a keyword as a variable name

```
True = "yes"    # SyntaxError: cannot assign to True
```

CORRECT — Use a plain identifier

```
is_active = True
```

WRONG — Confusing = (assignment) with == (comparison)

```
if age = 30:    # SyntaxError
```

CORRECT

```
if age == 30:
    print("Thirty!")
```

WRONG — Forgetting that bool values are capitalised in Python

```
is_student = true    # NameError: name 'true' is not defined
```

CORRECT

```
is_student = True
```

WRONG — Assuming type() returns a simple string

```
if type(age) == "int":    # Always False – type() returns a class, not a string
```

CORRECT

```
if type(age) == int:
    print("It's an integer")
# Or more Pythonically:
if isinstance(age, int):
    print("It's an integer")
```

Cross References

- **Next concept:** [Lab_2_2_Operators.md] — Arithmetic and comparison operators
- **Related:** [Lab_2_3_StringOps_TypeConversion.md] — Working with strings; converting between types
- **Practice:** [Practice_Problems.md] — Problems 1, 3, 4
- **Quick reference:** [Revision_Sheet.md] — Key Syntax, Important Functions Table